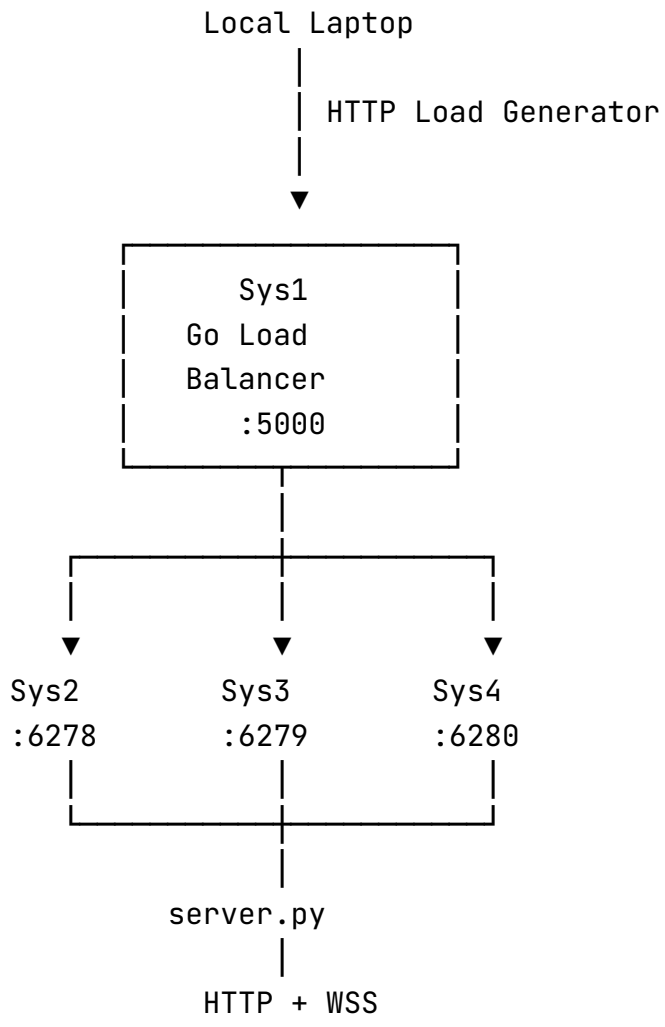
CSL599: LAB 5
LOAD BALANCER IN GO

NAME: PARITOSH LAHRE
ID: 12341550
ASSIGNED SYSTEMS:

- stu80_sys1 | 2277
- stu80_sys2 | 2278
- stu80_sys3 | 2279
- :stu80_sys4 | 2280

SYSTEM ARCHITECTURE:



CODES:

[SERVER.PY]

- I have added http service on the chat application backend (which works on websocket)
- Now backend is hosting both (http and wss)
- Ports wss: 5000, http: 6000 (default)

(Code blocks for the http addition implementation within wss setup)

```
#------------------------------------------------------------
#load-balancer
#------------------------------------------------------------
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from urllib.parse import urlparse, parse_qs
import threading
import socket
import time
```

```
#------------load_balancer----HTTP-setup-----------------
HTTP_HOST = "0.0.0.0"
HTTP_PORT = int(os.getenv("HTTP_PORT", "6000"))

BACKEND_ID = os.getenv(
   "BACKEND_ID",
   socket.gethostname()
)
```

```
# ------------------------------------------------------------
# HTTP interface for Load Balancer experiment
# ------------------------------------------------------------

class HTTPHandler(BaseHTTPRequestHandler):

   def log_message(self, format, *args):
       """Keep HTTP logging concise."""
```

```
        print(
            f"[HTTP] {self.client_address[0]} - {format % args}"
        )

    def send_json(self, status_code, payload):
        """Send a JSON HTTP response."""

        body = json.dumps(payload).encode("utf-8")

        self.send_response(status_code)

        self.send_header(
            "Content-Type",
            "application/json"
        )

        self.send_header(
            "Content-Length",
            str(len(body))
        )

        self.send_header(
            "Access-Control-Allow-Origin",
            "*"
        )

        self.end_headers()

        self.wfile.write(body)

    def do_GET(self):
        """Handle HTTP GET requests."""

        parsed = urlparse(self.path)

        path = parsed.path
        query = parse_qs(parsed.query)
```

```
        # ------------------------------------------------
        # Health check
        # ------------------------------------------------

        if path == "/health":

            self.send_json(
                200,
                {
                    "status": "healthy",
                    "backend": BACKEND_ID,
                    "service": "messaging-backend"
                }
            )

            return

        # ------------------------------------------------
        # Workload endpoint
        # ------------------------------------------------

        if path == "/":

            # Optional synthetic delay.
            delay = 0

            try:
                delay_values = query.get("delay", [])

                if delay_values:
                    delay = float(delay_values[0])

                # Prevent accidentally huge delays.
                delay = max(0, min(delay, 30))

            except (ValueError, TypeError):
```

```
            delay = 0

        if delay > 0:
            time.sleep(delay)

        # Optional synthetic failure.
        fail_values = query.get("fail", [])

        should_fail = (
            fail_values
            and fail_values[0].lower() == "true"
        )

        if should_fail:

            self.send_json(
                500,
                {
                    "status": "error",
                    "backend": BACKEND_ID,
                    "message": "Synthetic backend failure"
                }
            )

            return

        self.send_json(
            200,
            {
                "status": "ok",
                "backend": BACKEND_ID,
                "message": "Response from messaging backend"
            }
        )

        return
```

```
        # ------------------------------------------------
        # Backend information
        # ------------------------------------------------

        if path == "/info":

            self.send_json(
                200,
                {
                    "backend": BACKEND_ID,
                    "hostname": socket.gethostname(),
                    "websocket_port": PORT,
                    "http_port": HTTP_PORT
                }
            )

            return

        # ------------------------------------------------
        # Not found
        # ------------------------------------------------

        self.send_json(
            404,
            {
                "status": "error",
                "message": "Endpoint not found"
            }
        )


def start_http_server():
    """Start the HTTP interface in a background thread."""

    http_server = ThreadingHTTPServer(
        (HTTP_HOST, HTTP_PORT),
        HTTPHandler
```

```
    )

    print(
        f"HTTP backend running on "
        f"http://{HTTP_HOST}:{HTTP_PORT}"
    )

    print(
        f"Backend ID: {BACKEND_ID}"
    )

    http_server.serve_forever()
```

```
    # ------------------------------------------------------
    # Start HTTP server in background thread
    # ------------------------------------------------------

    http_thread = threading.Thread(
        target=start_http_server,
        daemon=True
    )

    http_thread.start()
```

(within the main func, http thread initiates along with wss)

LOAD_BALANCER.GO

```
package main

import (
    "encoding/json"
    "flag"
    "fmt"
    "log"
```

```
    "net/http"
    "net/http/httputil"
    "net/url"
    "strings"
    "sync"
    "sync/atomic"
    "time"
)

// ------------------------------------------------------------
// Backend
// ------------------------------------------------------------

type Backend struct {
    URL   *url.URL `json:"url"`
    Alive bool     `json:"alive"`

    mu sync.RWMutex
}

func (b *Backend) IsAlive() bool {
    b.mu.RLock()
    defer b.mu.RUnlock()

    return b.Alive
}

func (b *Backend) SetAlive(alive bool) {
    b.mu.Lock()
    b.Alive = alive
    b.mu.Unlock()
}

// ------------------------------------------------------------
// Metrics
// ------------------------------------------------------------
```

```go
type Metrics struct {
    TotalRequests   atomic.Uint64
    Successful      atomic.Uint64
    Failed          atomic.Uint64
    BackendErrors   atomic.Uint64

    mu        sync.Mutex
    Latencies []time.Duration

    StartTime time.Time
}

func NewMetrics() *Metrics {
    return &Metrics{
        Latencies: make([]time.Duration, 0, 10000),
        StartTime: time.Now(),
    }
}

func (m *Metrics) Record(
    success bool,
    backendError bool,
    latency time.Duration,
) {
    m.TotalRequests.Add(1)

    if success {
        m.Successful.Add(1)
    } else {
        m.Failed.Add(1)
    }

    if backendError {
        m.BackendErrors.Add(1)
    }

    m.mu.Lock()
```

```
    m.Latencies = append(
        m.Latencies,
        latency,
    )

    m.mu.Unlock()
}

func percentile(
    values []time.Duration,
    percent float64,
) time.Duration {

    if len(values) == 0 {
        return 0
    }

    sorted := make([]time.Duration, len(values))

    copy(sorted, values)

    // Simple insertion sort.
    // Fine for assignment-scale measurements.
    for i := 1; i < len(sorted); i++ {

        key := sorted[i]

        j := i - 1

        for j >= 0 && sorted[j] > key {

            sorted[j+1] = sorted[j]

            j--
        }
```

```
        sorted[j+1] = key
    }

    index := int(
        (percent / 100.0) *
            float64(len(sorted)-1),
    )

    return sorted[index]
}

// ------------------------------------------------------------
// Load Balancer
// ------------------------------------------------------------

type LoadBalancer struct {
    backends []*Backend

    next uint64

    metrics *Metrics

    client *http.Client
}

func NewLoadBalancer(
    backendURLs []string,
    timeout time.Duration,
) (*LoadBalancer, error) {

    backends := make([]*Backend, 0)

    for _, rawURL := range backendURLs {

        rawURL = strings.TrimSpace(rawURL)

        if rawURL == "" {
```

```
            continue
        }

        parsed, err := url.Parse(rawURL)

        if err != nil {
            return nil, fmt.Errorf(
                "invalid backend URL %q: %w",
                rawURL,
                err,
            )
        }

        backends = append(
            backends,
            &Backend{
                URL:   parsed,
                Alive: false,
            },
        )
    }

    if len(backends) == 0 {
        return nil, fmt.Errorf(
            "no backend servers configured",
        )
    }

    return &LoadBalancer{
        backends: backends,

        metrics: NewMetrics(),

        client: &http.Client{
            Timeout: timeout,
        },
    }, nil
```

```go
}

// ------------------------------------------------------------
// Round Robin
// ------------------------------------------------------------

func (lb *LoadBalancer) nextBackend() *Backend {

   count := len(lb.backends)

   if count == 0 {
       return nil
   }

   start := atomic.AddUint64(
       &lb.next,
       1,
   ) - 1

   for i := 0; i < count; i++ {

       index := int(
           (start + uint64(i)) %
               uint64(count),
       )

       backend := lb.backends[index]

       if backend.IsAlive() {
           return backend
       }
   }

   return nil
}

// ------------------------------------------------------------
```

```go
// Health Checks
// ------------------------------------------------------------

func (lb *LoadBalancer) checkBackend(
    backend *Backend,
) {

    healthURL := *backend.URL

    healthURL.Path = "/health"

    request, err := http.NewRequest(
        http.MethodGet,
        healthURL.String(),
        nil,
    )

    if err ≠ nil {
        backend.SetAlive(false)
        return
    }

    response, err := lb.client.Do(request)

    if err ≠ nil {

        if backend.IsAlive() {
            log.Printf(
                "[HEALTH] %s → DOWN: %v",
                backend.URL,
                err,
            )
        }

        backend.SetAlive(false)

        return
```

```
    }

    defer response.Body.Close()

    healthy := response.StatusCode == http.StatusOK

    previous := backend.IsAlive()

    backend.SetAlive(healthy)

    if healthy && !previous {

        log.Printf(
            "[HEALTH] %s → UP",
            backend.URL,
        )

    } else if !healthy && previous {

        log.Printf(
            "[HEALTH] %s → DOWN (HTTP %d)",
            backend.URL,
            response.StatusCode,
        )
    }
}

func (lb *LoadBalancer) healthChecker(
    interval time.Duration,
) {

    ticker := time.NewTicker(interval)

    defer ticker.Stop()

    for {
```

```
        for _, backend := range lb.backends {
            go lb.checkBackend(backend)
        }

        <-ticker.C
    }
}

// ------------------------------------------------------------
// Reverse Proxy
// ------------------------------------------------------------

func (lb *LoadBalancer) proxyHandler(
    w http.ResponseWriter,
    r *http.Request,
) {

    start := time.Now()

    backend := lb.nextBackend()

    if backend == nil {

        lb.metrics.Record(
            false,
            true,
            time.Since(start),
        )

        http.Error(
            w,
            "No healthy backend available",
            http.StatusServiceUnavailable,
        )

        return
    }
```

```
target := *backend.URL

proxy := httputil.NewSingleHostReverseProxy(
    &target,
)

originalDirector := proxy.Director

proxy.Director = func(req *http.Request) {

    originalDirector(req)

    req.Header.Set(
        "X-Load-Balancer",
        "Go-Load-Balancer",
    )

    req.Header.Set(
        "X-Forwarded-Host",
        r.Host,
    )

    req.Header.Set(
        "X-Backend",
        backend.URL.Host,
    )
}

// Custom error handler handles backend
// connection failures and timeouts.
proxy.ErrorHandler = func(
    rw http.ResponseWriter,
    req *http.Request,
    err error,
) {
```

```
        log.Printf(
            "[PROXY ERROR] backend=%s error=%v",
            backend.URL,
            err,
        )

        backend.SetAlive(false)

        lb.metrics.Record(
            false,
            true,
            time.Since(start),
        )

        http.Error(
            rw,
            "Backend unavailable",
            http.StatusBadGateway,
        )
    }

    proxy.ModifyResponse = func(
        response *http.Response,
    ) error {

        latency := time.Since(start)

        success :=
            response.StatusCode ⩾ 200 &&
                response.StatusCode < 400

        backendError :=
            response.StatusCode ⩾ 500

        lb.metrics.Record(
            success,
            backendError,
```

```
            latency,
        )

        response.Header.Set(
            "X-Load-Balancer-Backend",
            backend.URL.Host,
        )

        return nil
    }

    proxy.ServeHTTP(w, r)
}

// ------------------------------------------------------------
// /lb/health
// ------------------------------------------------------------

func (lb *LoadBalancer) healthHandler(
    w http.ResponseWriter,
    r *http.Request,
) {

    healthy := false

    for _, backend := range lb.backends {

        if backend.IsAlive() {
            healthy = true
            break
        }
    }

    w.Header().Set(
        "Content-Type",
        "application/json",
    )
```

```
    if !healthy {

        w.WriteHeader(
            http.StatusServiceUnavailable,
        )

        json.NewEncoder(w).Encode(
            map[string]interface{}{
                "status": "unhealthy",
            },
        )

        return
    }

    json.NewEncoder(w).Encode(
        map[string]interface{}{
            "status": "healthy",
        },
    )
}

// ------------------------------------------------------------
// /lb/status
// ------------------------------------------------------------

func (lb *LoadBalancer) statusHandler(
    w http.ResponseWriter,
    r *http.Request,
) {

    type backendStatus struct {
        URL     string `json:"url"`
        Healthy bool   `json:"healthy"`
    }
```

```
    status := make(
        []backendStatus,
        0,
        len(lb.backends),
    )

    for _, backend := range lb.backends {

        status = append(
            status,
            backendStatus{
                URL: backend.URL.String(),
                Healthy: backend.IsAlive(),
            },
        )
    }

    w.Header().Set(
        "Content-Type",
        "application/json",
    )

    json.NewEncoder(w).Encode(
        map[string]interface{}{
            "backends": status,
        },
    )
}

// ------------------------------------------------------------
// /lb/metrics
// ------------------------------------------------------------

func (lb *LoadBalancer) metricsHandler(
    w http.ResponseWriter,
    r *http.Request,
) {
```

```
total :=
    lb.metrics.TotalRequests.Load()

successful :=
    lb.metrics.Successful.Load()

failed :=
    lb.metrics.Failed.Load()

backendErrors :=
    lb.metrics.BackendErrors.Load()

lb.metrics.mu.Lock()

latencies := append(
    []time.Duration(nil),
    lb.metrics.Latencies...,
)

lb.metrics.mu.Unlock()

elapsed := time.Since(
    lb.metrics.StartTime,
).Seconds()

rps := float64(0)

if elapsed > 0 {
    rps = float64(total) / elapsed
}

dropout := float64(0)

if total > 0 {
    dropout =
        float64(failed) /
```

```
                float64(total) *
                100
    }

    result := map[string]interface{}{
        "total_requests":   total,
        "successful":       successful,
        "failed":           failed,
        "backend_errors":   backendErrors,
        "throughput_rps":   rps,
        "dropout_percent":  dropout,
        "latency_samples": len(latencies),
        "p50_ms": float64(
            percentile(latencies, 50),
        ) / float64(time.Millisecond),
        "p95_ms": float64(
            percentile(latencies, 95),
        ) / float64(time.Millisecond),
        "p99_ms": float64(
            percentile(latencies, 99),
        ) / float64(time.Millisecond),
    }

    w.Header().Set(
        "Content-Type",
        "application/json",
    )

    json.NewEncoder(w).Encode(result)
}

// ---------------------------------------------------------
// Main
// ---------------------------------------------------------

func main() {
```

```
var (
    listenAddr = flag.String(
        "listen",
        ":8080",
        "Load balancer listen address",
    )

    backendList = flag.String(
        "backends",
        "",
        "Comma-separated backend URLs",
    )

    timeout = flag.Duration(
        "timeout",
        5*time.Second,
        "Backend request timeout",
    )

    healthInterval = flag.Duration(
        "health-interval",
        2*time.Second,
        "Backend health check interval",
    )
)

flag.Parse()

if *backendList == "" {

    log.Fatal(
        "Use -backends to specify backend URLs",
    )
}

rawBackends :=
    strings.Split(
```

```
        *backendList,
        ",",
    )

lb, err := NewLoadBalancer(
    rawBackends,
    *timeout,
)

if err != nil {
    log.Fatal(err)
}

// Initial health check.
for _, backend := range lb.backends {
    lb.checkBackend(backend)
}

// Background health checking.
go lb.healthChecker(
    *healthInterval,
)

mux := http.NewServeMux()

mux.HandleFunc(
    "/lb/health",
    lb.healthHandler,
)

mux.HandleFunc(
    "/lb/status",
    lb.statusHandler,
)

mux.HandleFunc(
    "/lb/metrics",
```

```
    lb.metricsHandler,
)

// Everything else goes through the reverse proxy.
mux.HandleFunc(
    "/",
    lb.proxyHandler,
)

server := &http.Server{
    Addr: *listenAddr,

    Handler: mux,

    ReadHeaderTimeout: 5 * time.Second,

    WriteTimeout: 10 * time.Second,

    IdleTimeout: 30 * time.Second,
}

log.Printf(
    "Load balancer listening on %s",
    *listenAddr,
)

log.Printf(
    "Backends: %s",
    *backendList,
)

log.Printf(
    "Health interval: %s",
    *healthInterval,
)

log.Printf(
```

```
        "Backend timeout: %s",
        *timeout,
    )

    if err := server.ListenAndServe(); err ≠ nil {

        if err ≠ http.ErrServerClosed {
            log.Fatal(err)
        }
    }
}
```

COMPARISON TABLE:

Experiment	Success	Failed	RPS	Dropout	p95	p99
1 Backend	1000	0	247.67	0.00%	1108.59 ms	2248.44 ms
3 Backends	1000	0	208.07	0.00%	1011.31 ms	1205.93 ms

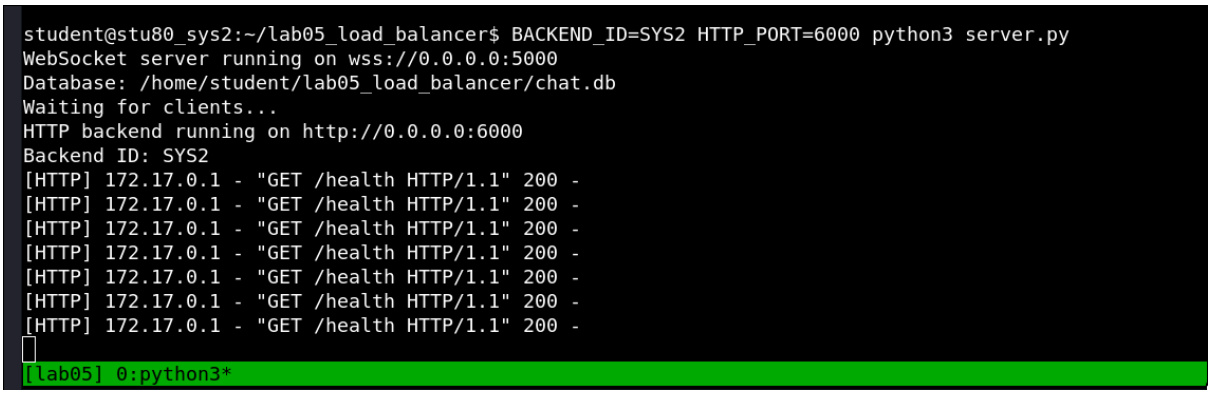
Load Generator:

- requests 1000
- concurrency 50
- timeout 10s

RELEVANT SCREENSHOTS:



- In the above screenshot we can observe how the load balancer detects failure of any backends and print logs. (up and down of servers)



Interval 2s
Timeout 5s